



# LogikaMobile

## Manual Corporativo de Operaciones, Identidad y Estándares de Ingeniería

---

### Documento Técnico Institucional

Versión: 2.0 (Expansión de Decálogo y Git Flow)

Fecha de Emisión: Junio 2026

Clasificación: Confidencial - Uso Interno Estricto

# 1. Identidad Visual y Uso de Marca

La consistencia visual de LogikaMobile refleja nuestro rigor metodológico. Toda propiedad digital (aplicaciones, webs, repositorios) y documentación (propuestas, manuales) debe adherirse estrictamente a estas especificaciones.

## 1.1 Paleta de Colores Institucionales

| Función         | Nombre     | Código HEX | Aplicación Estándar                                                  |
|-----------------|------------|------------|----------------------------------------------------------------------|
| Base Primaria   | Negro      | #000000    | Fondos principales, Dark Mode, tipografía de alto contraste.         |
| Base Identidad  | Morado     | #7B2CBF    | Títulos, branding corporativo, componentes de alta jerarquía.        |
| Base Acción     | Naranja    | #FF7A00    | Botones (CTA), alertas críticas, énfasis de conversión comercial.    |
| Acento Técnico  | Primario   | #347B6D    | Bordes, validaciones exitosas (Success states).                      |
| Acento Interfaz | Secundario | #6CD3D3    | Líneas de flujo en diagramas, loaders, acentos visuales secundarios. |

# 2. El Decálogo Operativo: Ciclo de Vida del Proyecto

Este decálogo es inquebrantable. Define la transición desde la abstracción de un problema de negocio hasta la ejecución material de código en producción. Omitir un paso resulta en deuda técnica y desviación de alcance.

## Fase I: Análisis y Definición Arquitectónica

### 1. Escucha Activa y Captación de Requerimientos

Sesión de auditoría inicial. El objetivo es extraer el dolor operativo del cliente (cuellos de botella, pérdidas financieras). Se documentan los objetivos de negocio y, críticamente, los **anti-objetivos** (lo que el sistema NO hará) para acotar el alcance de forma tajante.

Entregable: Documento de Requerimientos de Alto Nivel (HLR).

## 2. Entendimiento y Mapeo de Procesos As-Is / To-Be

Se audita la operación actual del cliente. Se levantan diagramas de flujo que exponen dependencias manuales y redundancias. Posteriormente, se diseña el flujo propuesto (To-Be) donde el software actuará como automatizador y validador, eliminando el error humano.

*Entregable: Diagramas de Flujo (BPMN / UML) del estado actual y futuro.*

## 3. Propuesta Tecnológica, Económica y Arquitectónica

Traducción de los diagramas en un stack tecnológico concreto (ej. Next.js, Ktor, PostgreSQL). Se elabora el presupuesto desglosado por módulos de desarrollo, tiempos de ejecución rígidos (Gantt) y proyecciones de SLA (Service Level Agreement). Se define el ROI para el cliente.

*Entregable: Propuesta Comercial y Documento de Arquitectura de Software (SAD).*

# Fase II: Diseño y Lógica de Negocio

## 4. Construcción de Reglas de Negocio (Cero Ambigüedad)

El código no asume; ejecuta. Se redactan las reglas operativas bajo estructuras lógicas estrictas (Given-When-Then). Se definen permisos, roles, validaciones de campos, límites transaccionales y flujos de error. Si un escenario no está en este documento, no se programa.

*Entregable: Matriz de Reglas de Negocio / Historias de Usuario Detalladas.*

## 5. Modelado de Datos y Pruebas de Concepto (POC)

Se construye el Diagrama Entidad-Relación (ERD) asegurando la normalización de la base de datos hasta la 3FN. A la par, se diseñan interfaces (Mockups en Figma) y, si el proyecto requiere integraciones críticas (ej. pasarelas de pago o hardware), se aísla y programa una POC técnica.

*Entregable: Diagrama ERD, Mockups UI/UX de alta fidelidad, Repositorio POC.*

# Fase III: Ejecución de Ingeniería

## 6. Codificación Limpia y Mantenible

Ejecución del desarrollo basada en patrones de diseño arquitectónico (Clean Architecture, MVVM). Cumplimiento estricto de principios SOLID y DRY. El código debe ser autodescriptivo. Los Pull Requests exigen revisión de pares (Code Review) antes de integrarse a la rama de desarrollo.

*Entregable: Repositorio versionado con feature-branches funcionales.*

## 7. Pruebas Automatizadas y Depuración Continua

El QA no es una fase final, es continuo. Todo módulo de lógica de negocio o cálculo financiero exige cobertura mediante pruebas unitarias. Los endpoints de la API deben someterse a pruebas de integración. Los fallos se detectan localmente, nunca en producción.

*Entregable: Reporte de cobertura de código (Test Coverage > 80%).*

## Fase IV: Estabilización y Entrega

### 8. Estabilización de Versión Demo (Staging)

El código se fusiona hacia la rama de desarrollo (Staging). Se aplica un "Code Freeze" donde no se aceptan nuevas características. Se ejecutan pruebas de regresión y estrés para garantizar que la infraestructura soporta las proyecciones de carga iniciales.

*Entregable: Entorno de Staging operativo y estable.*

### 9. Presentación de Demo y User Acceptance Testing (UAT)

Sesión de validación final con el cliente. Se opera el sistema en vivo simulando escenarios reales de negocio. El cliente firma la aceptación formal de que el software cumple exhaustivamente con las reglas de negocio estipuladas en el Paso 4.

*Entregable: Acta de aceptación y liberación (Sign-off).*

### 10. Despliegue a Producción y Observabilidad

Pase a producción automatizado (Pipeline CI/CD). Una vez el tráfico es enrutado al nuevo servidor, se activan herramientas de Application Performance Monitoring (APM). Se vigila el uptime, latencias de red y consumo de memoria para una intervención preventiva, no reactiva.

*Entregable: Sistema en Producción, Dashboard de Monitoreo activo.*

## 3. Estándares de Control de Versiones (Git Flow)

Para asegurar despliegues predecibles y trazabilidad de errores, LogikaMobile opera bajo una política restrictiva de control de ramas y commits.

### 3.1 Entornos y Ramas Principales

- **Main / Master:** Representa el entorno de PRODUCCIÓN. El código aquí es inmutable manualmente; solo recibe actualizaciones (merges) provenientes de un Release o Hotfix previamente probado.
- **Development:** Representa el entorno de DESARROLLO (Staging/QA). Centraliza todas las características terminadas antes de empaquetarse para una versión productiva.

### 3.2 Nomenclatura de Ramas de Trabajo

Toda nueva línea de código debe originarse en una rama aislada con la siguiente estructura:

```
dev/uh-xxx/<tipo>/<developer>
```

- **uh-xxx** : Identificador del ticket o User History (ej. uh-042).
- **<tipo>** : Debe ser estrictamente **fix** (para reparación de errores) o **feat** (para nuevas funcionalidades).
- **<developer>** : Nombre o iniciales del ingeniero a cargo (ej. luis).

Ejemplo válido: `dev/uh-014/feat/luis`

Ejemplo válido: `dev/uh-089/fix/luis`

### 3.3 Estructura Estricta de Commits

El historial de Git es documentación técnica. Queda prohibido el uso de mensajes genéricos ("fix bug", "changes"). Cada commit debe seguir la siguiente plantilla exacta:

```
[DEVELOPER] [UH-XXX] [TIPO] MENSAJE DESCRIPTIVO
```

- **[DEVELOPER]**: Identificador del autor (ej. [LUIS]).
- **[UH-XXX]**: Ticket asociado (ej. [UH-014]).
- **[TIPO]**: **FIX** (reparación), **REFACTOR** (mejora sin cambio de lógica), o **FEATURE** (nueva funcionalidad).
- **MENSAJE**: Descripción imperativa y concisa de la mutación realizada.

Ejemplo correcto:

```
[LUIS][UH-014][FEATURE] Implementacion de middleware de autenticacion JWT en Ktor
```

```
[LUIS][UH-089][FIX] Resolucion de NullPointerException en modulo de cotizaciones
```

### 3.4 Pipeline CI/CD Obligatorio

Todo Merge Request hacia **Development** o **Main** disparará GitHub Actions validando:

1. Análisis estático de código (Linter).
2. Ejecución total de la suite de pruebas automatizadas.
3. Revisión de vulnerabilidades de dependencias.

Si algún proceso falla, el Merge Request queda bloqueado automáticamente.